

Scope of Work: Hugging Trees Airtable Player Modernization

Table of Contents

- [Project Overview](#)
- [Team Structure](#)
- [Methodology: Technical Kanban](#)
- [Kanban Board Columns](#)
- [Key Principles](#)
- [User Stories](#)
- [US-001: Custom Airtable Wrapper](#)
- [Feature US-001: Custom Airtable Wrapper](#)
- [US-002: Developer Installation and Contribution Guide](#)
- [Feature US-002: Developer Installation and Contribution Guide](#)
- [US-003: Automated Test Coverage](#)
- [Feature US-003: Automated Test Coverage](#)
- [US-004: Cross-Platform Developer Workflow](#)
- [Feature US-004: Cross-Platform Developer Workflow](#)
- [US-005: React Application Foundation](#)
- [Feature US-005: React Application Foundation](#)
- [US-006: Storybook and Jest Integration](#)
- [Feature US-006: Storybook and Jest Integration](#)
- [US-007: Airtable-Driven React Configuration](#)
- [Feature US-007: Airtable-Driven React Configuration](#)
- [Deliverables](#)
- [Delivery Estimate Summary](#)
- [Milestones](#)
- [Pricing Estimate](#)
- [Assumptions and Constraints](#)

- Risk Management

Key	Description
Client	Hugging Trees
Project Period	2026-04-01 to 2026-07-31

Project Overview

Rebuild and fine-tune the existing Airtable-driven static media experience as a React application with a custom Airtable wrapper, configurable UI behavior, cross-platform developer workflows, automated tests, and maintainable contributor documentation.

Team Structure

Role	Name	Email	Phone
Project Manager	Jordan Reeves	jordan.reeves@abccorp.example	+1-312-555-0142
Senior Developer	Priya Shah	priya.shah@abccorp.example	+1-312-555-0168
UI/UX Designer	Avery Collins	avery.collins@abccorp.example	+1-312-555-0185
QA Engineer	Marcus Bennett	marcus.bennett@abccorp.example	+1-312-555-0131

Methodology: Technical Kanban

This engagement will follow a technical Kanban workflow so the team can modernize the application iteratively while keeping Airtable integration, UI, and deployment work visible.

Kanban Board Columns

Key	Description
Backlog	Approved modernization work waiting for prioritization, refinement, or sequencing.
Ready	Tasks that have enough detail, acceptance criteria, and dependencies cleared for implementation.

Key	Description
In Progress	Active engineering, design, integration, or QA work underway for the React rebuild.
Review	Completed work awaiting peer review, stakeholder validation, or functional QA signoff.
Done	Accepted work that is validated, documented, and ready for release or handoff.

Key Principles

Key	Description
Visualize workflow	Track frontend, Airtable integration, documentation, and QA work in one shared delivery view.
Limit work in progress (WIP)	Reduce parallel changes so the team can stabilize the React rewrite and wrapper layer effectively.
Manage flow and reduce handoff delays	Monitor bottlenecks between implementation, review, QA, and content validation.
Make process policies explicit	Define code review, testing, and Airtable schema change expectations before work starts.
Use feedback loops to refine delivery	Review working increments with the client regularly and adjust scope priorities as the app evolves.
Improve collaboratively and continuously	Use short retrospectives and release feedback to improve delivery quality over the engagement.

User Stories

US-001: Custom Airtable Wrapper

User Story:

- As a developer
- I want to create a custom Airtable wrapper
- So that I can CRUD the database from a custom application

Feature US-001: Custom Airtable Wrapper

AC ID	US ID	Acceptance Criteria
AC-01	US-001	The wrapper supports authenticated create, read, update, and delete operations for the target Airtable base.
AC-02	US-001	The wrapper centralizes Airtable schema mappings and request handling for reuse across the app.
AC-03	US-001	Errors, rate limits, and invalid payloads are surfaced with actionable logs and user-safe responses.
AC-04	US-001	The wrapper can be consumed by the React application without exposing secrets in the client.

Code Block

Feature: Custom Airtable Wrapper
Scenario: The wrapper supports authenticated create, read, update, and delete operations for the ta
Given a developer
When they need to create a custom Airtable wrapper
Then they can CRUD the database from a custom application

Code Block

Feature: Custom Airtable Wrapper
Scenario: The wrapper centralizes Airtable schema mappings and request handling for reuse across th
Given a developer
When they need to create a custom Airtable wrapper
Then they can CRUD the database from a custom application

Code Block

Feature: Custom Airtable Wrapper
Scenario: Errors, rate limits, and invalid payloads are surfaced with actionable logs and user-safe
Given a developer
When they need to create a custom Airtable wrapper
Then they can CRUD the database from a custom application

Code Block

Feature: Custom Airtable Wrapper
Scenario: The wrapper can be consumed by the React application without exposing secrets in the clie
Given a developer
When they need to create a custom Airtable wrapper
Then they can CRUD the database from a custom application

Key	Description
Estimate	13 story points
Estimated Hours	104 hours
Priority	High

US-002: Developer Installation and Contribution Guide

User Story:

- As a developer
- I want installation, contribution, and documentation guidelines for the application
- So that I can help new contributors set up, understand, and extend the project efficiently

Feature US-002: Developer Installation and Contribution Guide

AC ID	US ID	Acceptance Criteria
AC-01	US-002	The repository includes installation steps for local development and production-oriented builds.
AC-02	US-002	Contribution guidance defines branching, code standards, review expectations, and release workflow.
AC-03	US-002	Architecture and Airtable integration documentation explain how the app is organized and configured.
AC-04	US-002	Documentation is versioned with the codebase and updated as part of feature delivery.

Code Block

Feature: Developer Installation and Contribution Guide
Scenario: The repository includes installation steps for local development and production-oriented
Given a developer
When they need installation, contribution, and documentation guidelines for the application
Then they can help new contributors set up, understand, and extend the project efficiently

Code Block

Feature: Developer Installation and Contribution Guide
Scenario: Contribution guidance defines branching, code standards, review expectations, and release
Given a developer
When they need installation, contribution, and documentation guidelines for the application
Then they can help new contributors set up, understand, and extend the project efficiently

Code Block

Feature: Developer Installation and Contribution Guide
Scenario: Architecture and Airtable integration documentation explain how the app is organized and
Given a developer
When they need installation, contribution, and documentation guidelines for the application
Then they can help new contributors set up, understand, and extend the project efficiently

Code Block

Feature: Developer Installation and Contribution Guide
Scenario: Documentation is versioned with the codebase and updated as part of feature delivery.
Given a developer
When they need installation, contribution, and documentation guidelines for the application
Then they can help new contributors set up, understand, and extend the project efficiently

Key	Description
Estimate	5 story points
Estimated Hours	40 hours
Priority	Medium

US-003: Automated Test Coverage

User Story:

- As a developer
- I want tests for the application
- So that I can validate changes reliably as the app is rebuilt and fine-tuned

Feature US-003: Automated Test Coverage

AC ID	US ID	Acceptance Criteria
AC-01	US-003	The project includes unit and integration test coverage for the Airtable wrapper and core UI behavior.
AC-02	US-003	Tests run in a repeatable local and CI-friendly command.
AC-03	US-003	Critical user flows are covered for data loading, error handling, and key UI interactions.
AC-04	US-003	Test documentation explains how to run, extend, and troubleshoot the test suite.

Code Block

```
Feature: Automated Test Coverage
  Scenario: The project includes unit and integration test coverage for the Airtable wrapper and core
    Given a developer
    When they need tests for the application
    Then they can validate changes reliably as the app is rebuilt and fine-tuned
```

Code Block

```
Feature: Automated Test Coverage
  Scenario: Tests run in a repeatable local and CI-friendly command.
    Given a developer
    When they need tests for the application
    Then they can validate changes reliably as the app is rebuilt and fine-tuned
```

Code Block

```
Feature: Automated Test Coverage
```

Scenario: Critical user flows are covered for data loading, error handling, and key UI interactions
Given a developer
When they need tests for the application
Then they can validate changes reliably as the app is rebuilt and fine-tuned

Code Block

Feature: Automated Test Coverage
Scenario: Test documentation explains how to run, extend, and troubleshoot the test suite.
Given a developer
When they need tests for the application
Then they can validate changes reliably as the app is rebuilt and fine-tuned

Key	Description
Estimate	8 story points
Estimated Hours	64 hours
Priority	High

US-004: Cross-Platform Developer Workflow

User Story:

- As a developer
- I want the application to be platform agnostic with installation steps and scripts
- So that I can support contributors on Windows, macOS, and Linux consistently

Feature US-004: Cross-Platform Developer Workflow

AC ID	US ID	Acceptance Criteria
AC-01	US-004	Setup instructions cover Windows, macOS, and Linux prerequisites and environment configuration.
AC-02	US-004	Development scripts work across supported platforms or include documented platform-specific equivalents.
AC-03	US-004	Build and test commands are standardized for contributors and CI environments.
AC-04	US-004	Platform-specific issues are documented with mitigation steps or automation where practical.

Code Block

Feature: Cross-Platform Developer Workflow
Scenario: Setup instructions cover Windows, macOS, and Linux prerequisites and environment configur
Given a developer

When they need the application to be platform agnostic with installation steps and scripts
Then they can support contributors on Windows, macOS, and Linux consistently

Code Block

Feature: Cross-Platform Developer Workflow

Scenario: Development scripts work across supported platforms or include documented platform-specific

Given a developer

When they need the application to be platform agnostic with installation steps and scripts

Then they can support contributors on Windows, macOS, and Linux consistently

Code Block

Feature: Cross-Platform Developer Workflow

Scenario: Build and test commands are standardized for contributors and CI environments.

Given a developer

When they need the application to be platform agnostic with installation steps and scripts

Then they can support contributors on Windows, macOS, and Linux consistently

Code Block

Feature: Cross-Platform Developer Workflow

Scenario: Platform-specific issues are documented with mitigation steps or automation where practical

Given a developer

When they need the application to be platform agnostic with installation steps and scripts

Then they can support contributors on Windows, macOS, and Linux consistently

Key	Description
Estimate	5 story points
Estimated Hours	40 hours
Priority	Medium

US-005: React Application Foundation

User Story:

- As a product owner
- I want the application in React
- So that I can maintain and evolve a component-driven UI more easily

Feature US-005: React Application Foundation

AC ID	US ID	Acceptance Criteria
AC-01	US-005	The current static application experience is rebuilt as a React application.

AC ID	US ID	Acceptance Criteria
AC-02	US-005	Core screens and media flows are migrated into reusable React components.
AC-03	US-005	State management and data loading patterns are documented for future contributors.
AC-04	US-005	The React application integrates with the custom Airtable wrapper for data-driven functionality.

Code Block

Feature: React Application Foundation
 Scenario: The current static application experience is rebuilt as a React application.
 Given a product owner
 When they need the application in React
 Then they can maintain and evolve a component-driven UI more easily

Code Block

Feature: React Application Foundation
 Scenario: Core screens and media flows are migrated into reusable React components.
 Given a product owner
 When they need the application in React
 Then they can maintain and evolve a component-driven UI more easily

Code Block

Feature: React Application Foundation
 Scenario: State management and data loading patterns are documented for future contributors.
 Given a product owner
 When they need the application in React
 Then they can maintain and evolve a component-driven UI more easily

Code Block

Feature: React Application Foundation
 Scenario: The React application integrates with the custom Airtable wrapper for data-driven function
 Given a product owner
 When they need the application in React
 Then they can maintain and evolve a component-driven UI more easily

Key	Description
Estimate	13 story points
Estimated Hours	104 hours
Priority	High

US-006: Storybook and Jest Integration

User Story:

- As a developer
- I want Storybook and Jest in the React app
- So that I can develop, preview, and test components in isolation

Feature US-006: Storybook and Jest Integration

AC ID	US ID	Acceptance Criteria
AC-01	US-006	Storybook is configured for reusable React components with representative states.
AC-02	US-006	Jest is configured for component and logic tests with clear setup instructions.
AC-03	US-006	Key UI components have baseline stories and tests.
AC-04	US-006	The developer workflow includes commands for running Storybook and Jest locally.

Code Block

Feature: Storybook and Jest Integration
Scenario: Storybook is configured for reusable React components with representative states.
Given a developer
When they need Storybook and Jest in the React app
Then they can develop, preview, and test components in isolation

Code Block

Feature: Storybook and Jest Integration
Scenario: Jest is configured for component and logic tests with clear setup instructions.
Given a developer
When they need Storybook and Jest in the React app
Then they can develop, preview, and test components in isolation

Code Block

Feature: Storybook and Jest Integration
Scenario: Key UI components have baseline stories and tests.
Given a developer
When they need Storybook and Jest in the React app
Then they can develop, preview, and test components in isolation

Code Block

Feature: Storybook and Jest Integration
Scenario: The developer workflow includes commands for running Storybook and Jest locally.
Given a developer
When they need Storybook and Jest in the React app
Then they can develop, preview, and test components in isolation

Key	Description
Estimate	8 story points
Estimated Hours	64 hours
Priority	Medium

US-007: Airtable-Driven React Configuration

User Story:

- As a content administrator
- I want the React app to be configurable by Airtable
- So that I can adjust supported content and interface behavior without code changes

Feature US-007: Airtable-Driven React Configuration

AC ID	US ID	Acceptance Criteria
AC-01	US-007	Airtable can control defined configuration fields used by the React application.
AC-02	US-007	The app validates and safely handles missing or malformed Airtable configuration values.
AC-03	US-007	Configuration changes can update UI content, media references, or display behavior as designed.
AC-04	US-007	Documentation identifies which configuration fields are editable in Airtable and how they affect the app.

Code Block

```
Feature: Airtable-Driven React Configuration
  Scenario: Airtable can control defined configuration fields used by the React application.
    Given a content administrator
    When they need the React app to be configurable by Airtable
    Then they can adjust supported content and interface behavior without code changes
```

Code Block

```
Feature: Airtable-Driven React Configuration
  Scenario: The app validates and safely handles missing or malformed Airtable configuration values.
    Given a content administrator
    When they need the React app to be configurable by Airtable
    Then they can adjust supported content and interface behavior without code changes
```

Code Block

Feature: Airtable-Driven React Configuration
Scenario: Configuration changes can update UI content, media references, or display behavior as desired
Given a content administrator
When they need the React app to be configurable by Airtable
Then they can adjust supported content and interface behavior without code changes

Code Block

Feature: Airtable-Driven React Configuration
Scenario: Documentation identifies which configuration fields are editable in Airtable and how they are used
Given a content administrator
When they need the React app to be configurable by Airtable
Then they can adjust supported content and interface behavior without code changes

Key	Description
Estimate	8 story points
Estimated Hours	64 hours
Priority	High

Deliverables

Deliverable ID	Deliverable	Description	Access Details
D-01	Custom Airtable wrapper library	A reusable application layer for secure Airtable CRUD operations, schema mapping, and error handling.	Developers will access the wrapper through the project repository and the documented integration entry points.
D-02	React application rebuild	A React-based version of the Airtable player experience replacing the current static implementation.	Stakeholders will access the rebuilt application through the deployed environment and local developers through the documented run scripts.
D-03	Storybook and Jest setup	Component preview and automated test tooling configured for the React application.	The team will access Storybook locally or in a hosted preview and run Jest from the project scripts.
D-04	Cross-platform setup and contributor documentation	Installation, contribution, architecture, and workflow guidance for Windows, macOS, and Linux contributors.	Contributors will access this documentation in the repository README and supporting docs.
D-05	Airtable configuration model	Documented Airtable-driven settings that control supported areas of application behavior and content.	Content administrators will manage configuration in Airtable and reference the project documentation for supported fields.

Deliverable ID	Deliverable	Description	Access Details
D-06	Deployment and QA handoff package	Release scripts, validation steps, and handoff guidance for maintaining the application after delivery.	The client and support team will access the handoff package through the shared repository and delivery workspace.

Delivery Estimate Summary

Epic ID	Epic	Feature	Story Points	Estimated Hours	Priority
US-001	Custom Airtable Wrapper	Custom Airtable Wrapper	13	104	High
US-002	Developer Installation and Contribution Guide	Developer Installation and Contribution Guide	5	40	Medium
US-003	Automated Test Coverage	Automated Test Coverage	8	64	High
US-004	Cross-Platform Developer Workflow	Cross-Platform Developer Workflow	5	40	Medium
US-005	React Application Foundation	React Application Foundation	13	104	High
US-006	Storybook and Jest Integration	Storybook and Jest Integration	8	64	Medium
US-007	Airtable-Driven React Configuration	Airtable-Driven React Configuration	8	64	High
Total	-	-	60	480	-

Milestones

Milestone	Due Date	Deliverables
Discovery and Architecture Baseline	2026-04-15	Current app review and modernization plan Airtable wrapper design React architecture and delivery backlog
Core React and Airtable Integration	2026-05-15	Working React application foundation Initial Airtable wrapper implementation Configurable data loading flow

Milestone	Due Date	Deliverables
Quality, Documentation, and Platform Support	2026-06-15	Storybook and Jest setup Cross-platform install and contribution documentation Expanded automated test coverage
Final Delivery	2026-07-31	Production-ready React application Airtable-driven configuration support Deployment and QA handoff package

Pricing Estimate

Key	Description
Total Story Points	60
Estimated Hours	480 (assuming 8 hours per story point)
Hourly Rate	\$95.00
Base Cost	\$45,600.00
Overhead (25.0%)	\$11,400.00
Profit Margin (20.0%)	\$11,400.00
Total Estimated Cost	\$68,400.00

Assumptions and Constraints

ID	Summary	Mitigation
A-01	The existing static app and Airtable base can be reviewed to confirm data structure and current workflow expectations.	Schedule discovery early and capture schema, dependency, and hosting decisions before implementation begins.
A-02	Airtable API access, credentials, and target environments will be available to the delivery team when needed.	Track access dependencies as kickoff actions and escalate blocked credentials immediately.
A-03	The client will identify which UI and behavior settings should be Airtable-configurable in the first release.	Define a supported configuration scope during backlog refinement and defer nonessential variability to later phases.

ID	Summary	Mitigation
A-04	React, Storybook, and Jest are acceptable technology choices for the modernization effort.	Confirm the toolchain during project kickoff and document any required substitutions before implementation starts.

Risk Management

ID	Summary	Mitigation
R-01	Legacy static behaviors may not map cleanly to the React component model without discovery and refactoring.	Prototype the most complex media and Airtable flows early to reduce architectural uncertainty.
R-02	Airtable schema changes during implementation could affect wrapper and UI assumptions.	Centralize schema mapping in the wrapper and review base changes during each planning cycle.
R-03	Cross-platform setup can introduce environment-specific issues that slow onboarding.	Validate scripts on each supported platform and document troubleshooting steps as part of delivery.
R-04	Configuration-driven UI behavior can create unstable edge cases if unsupported Airtable values are introduced.	Define guardrails, validation, and fallback behavior for configurable fields before exposing them to content editors.